

InplaceFunction#

<https://pytorch.org/docs/stable/generated/torch.autograd.function.InplaceFunction.html>

Description:

This class is here only for backward compatibility reasons. Use `Function` instead of this for any new use case. Define a formula for differentiating the operation with backward mode automatic differentiation. This function is to be overridden by all subclasses. (Defining this function is equivalent to defining the `vjp` function.) It must accept a context `ctx` as the first argument, followed by as many outputs as the `forward()` returned (None will be passed in for non tensor outputs of the forward function), and it should return as many tensors, as there were inputs to `forward()`. Each argument is the gradient w.r.t the given output, and each returned value should be the gradient w.r.t. the corresponding input. If an input is not a Tensor or is a Tensor not requiring grads, you can just pass None as a gradient for that input. The context can be used to retrieve tensors saved during the forward pass. It also has an attribute `ctx.needs_input_grad` as a tuple of booleans representing whether each input needs gradient. E.g., `backward()` will have `ctx.needs_input_grad[0]` = True if the first input to `forward()` needs gradient computed w.r.t. the output.

Function Signature

```
class torch.autograd.function.InplaceFunction(inplace=False)
[source]#
```

```
static backward(ctx, *grad_outputs)[source]#
```

Return type

```
static forward(*args, **kwargs)[source]#
```

Return type

Returns:

Any

Code Examples

```
@staticmethod
def forward(ctx: Any, *args: Any, **kwargs: Any) -> Any:
    pass
```

```

@staticmethod
def forward(*args: Any, **kwargs: Any) -> Any:
    pass

@staticmethod
def setup_context(ctx: Any, inputs: Tuple[Any, ...], output:
Any) -> None:
    pass

```

```

>>> class Inplace(Function):
>>>     @staticmethod
>>>     def forward(ctx, x):
>>>         x_numpy = x.numpy() # x_numpy shares storage with x
>>>         x_numpy += 1
>>>         ctx.mark_dirty(x)
>>>         return x
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, grad_output):
>>>         return grad_output
>>>
>>> a = torch.tensor(1., requires_grad=True,
dtype=torch.double).clone()
>>> b = a * a
>>> Inplace.apply(a) # This would lead to wrong gradients!
>>>                 # but the engine would not know unless we
mark_dirty
>>> b.backward() # RuntimeError: one of the variables needed for
gradient
>>>             # computation has been modified by an inplace
operation

```

```
>>> class Func(Function):
>>>     @staticmethod
>>>     def forward(ctx, x):
>>>         sorted, idx = x.sort()
>>>         ctx.mark_non_differentiable(idx)
>>>         ctx.save_for_backward(x, idx)
>>>         return sorted, idx
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, g1, g2): # still need to accept g2
>>>         x, idx = ctx.saved_tensors
>>>         grad_input = torch.zeros_like(x)
>>>         grad_input.index_add_(0, idx, g1)
>>>         return grad_input
```

```

>>> class Func(Function):
>>>     @staticmethod
>>>     def forward(ctx, x: torch.Tensor, y: torch.Tensor, z:
int):
>>>         w = x * z
>>>         out = x * y + y * z + w * y
>>>         ctx.save_for_backward(x, y, w, out)
>>>         ctx.z = z # z is not a tensor
>>>         return out
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, grad_out):
>>>         x, y, w, out = ctx.saved_tensors
>>>         z = ctx.z
>>>         gx = grad_out * (y + y * z)
>>>         gy = grad_out * (x + z + w)
>>>         gz = None
>>>         return gx, gy, gz
>>>
>>> a = torch.tensor(1., requires_grad=True, dtype=torch.double)
>>> b = torch.tensor(2., requires_grad=True, dtype=torch.double)
>>> c = 4
>>> d = Func.apply(a, b, c)

```

```

>>> class Func(torch.autograd.Function):
>>>     @staticmethod
>>>     def forward(ctx, x: torch.Tensor, y: torch.Tensor, z:
int):
>>>         ctx.save_for_backward(x, y)
>>>         ctx.save_for_forward(x, y)
>>>         ctx.z = z
>>>         return x * y * z
>>>
>>>     @staticmethod
>>>     def jvp(ctx, x_t, y_t, _):
>>>         x, y = ctx.saved_tensors
>>>         z = ctx.z
>>>         return z * (y * x_t + x * y_t)
>>>
>>>     @staticmethod
>>>     def vjp(ctx, grad_out):
>>>         x, y = ctx.saved_tensors
>>>         z = ctx.z
>>>         return z * grad_out * y, z * grad_out * x, None
>>>
>>>     a = torch.tensor(1., requires_grad=True,
dtype=torch.double)
>>>     t = torch.tensor(1., dtype=torch.double)
>>>     b = torch.tensor(2., requires_grad=True,
dtype=torch.double)
>>>     c = 4
>>>
>>>     with fwAD.dual_level():
>>>         a_dual = fwAD.make_dual(a, t)
>>>         d = Func.apply(a_dual, b, c)

```

```

>>> class SimpleFunc(Function):
>>>     @staticmethod
>>>     def forward(ctx, x):
>>>         return x.clone(), x.clone()
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, g1, g2):
>>>         return g1 + g2  # No check for None necessary
>>>
>>> # We modify SimpleFunc to handle non-materialized grad
>>> outputs
>>> class Func(Function):
>>>     @staticmethod
>>>     def forward(ctx, x):
>>>         ctx.set_materialize_grads(False)
>>>         ctx.save_for_backward(x)
>>>         return x.clone(), x.clone()
>>>
>>>     @staticmethod
>>>     @once_differentiable
>>>     def backward(ctx, g1, g2):
>>>         x, = ctx.saved_tensors
>>>         grad_input = torch.zeros_like(x)
>>>         if g1 is not None:  # We must check for None now
>>>             grad_input += g1
>>>         if g2 is not None:
>>>             grad_input += g2
>>>         return grad_input
>>>
>>> a = torch.tensor(1., requires_grad=True)
>>> b, _ = Func.apply(a)  # induces g2 to be undefined

```

Detailed Documentation

Documentation

This class is here only for backward compatibility reasons. Use `Function` instead of this for any new use case.

Define a formula for differentiating the operation with backward mode automatic differentiation.

This function is to be overridden by all subclasses. (Defining this function is equivalent to defining the `vjp` function.)

It must accept a context `ctx` as the first argument, followed by as many outputs as the `forward()` returned (None will be passed in for non tensor outputs of the forward function), and it should return as many tensors, as there were inputs to `forward()`. Each argument is the gradient w.r.t the given output, and each returned value should be the gradient w.r.t. the corresponding input. If an input is not a Tensor or is a Tensor not requiring grads, you can just pass None as a gradient for that input.

The context can be used to retrieve tensors saved during the forward pass. It also has an attribute `ctx.needs_input_grad` as a tuple of booleans representing whether each input needs gradient. E.g., `backward()` will have `ctx.needs_input_grad[0] = True` if the first input to `forward()` needs gradient computed w.r.t. the output.

Define the forward of the custom autograd Function.

This function is to be overridden by all subclasses. There are two ways to define `forward`:

Usage 1 (Combined forward and `ctx`):

It must accept a context `ctx` as the first argument, followed by any number of arguments (tensors or other types).

See Combined or separate `forward()` and `setup_context()` for more details

Usage 2 (Separate forward and ctx):

The forward no longer accepts a ctx argument.

Instead, you must also override the `torch.autograd.Function.setup_context()` staticmethod to handle setting up the ctx object. output is the output of the forward, inputs are a Tuple of inputs to the forward.

See Extending `torch.autograd` for more details

The context can be used to store arbitrary data that can be then retrieved during the backward pass. Tensors should not be stored directly on ctx (though this is not currently enforced for backward compatibility). Instead, tensors should be saved either with `ctx.save_for_backward()` if they are intended to be used in backward (equivalently, vjp) or `ctx.save_for_forward()` if they are intended to be used for in jvp.

Define a formula for differentiating the operation with forward mode automatic differentiation.

This function is to be overridden by all subclasses. It must accept a context ctx as the first argument, followed by as many inputs as the `forward()` got (None will be passed in for non tensor inputs of the forward function), and it should return as many tensors as there were outputs to `forward()`. Each argument is the gradient w.r.t the given input, and each returned value should be the gradient w.r.t. the corresponding output. If an output is not a Tensor or the function is not differentiable with respect to that output, you can just pass None as a gradient for that input.

You can use the ctx object to pass any value from the forward to this functions.

Mark given tensors as modified in an in-place operation.

This should be called at most once, in either the `setup_context()` or `forward()` methods, and all arguments should be inputs.

Every tensor that's been modified in-place in a call to `forward()` should be given to this function, to ensure correctness of our checks. It doesn't matter whether the function is called before or after modification.

Mark outputs as non-differentiable.

This should be called at most once, in either the `setup_context()` or `forward()` methods, and all arguments should be tensor outputs.

This will mark outputs as not requiring gradients, increasing the efficiency of backward computation. You still need to accept a gradient for each output in `backward()`, but it's always going to be a zero tensor with the same shape as the shape of a corresponding output.

Save given tensors for a future call to `backward()`.

`save_for_backward` should be called at most once, in either the `setup_context()` or `forward()` methods, and only with tensors.

All tensors intended to be used in the backward pass should be saved with `save_for_backward` (as opposed to directly on `ctx`) to prevent incorrect gradients and memory leaks, and enable the application of saved tensor hooks. See `torch.autograd.graph.saved_tensors_hooks`. See [Extending torch.autograd](#) for more details.

Note that if intermediary tensors, tensors that are neither inputs nor outputs of `forward()`, are saved for backward, your custom Function may not support double backward. Custom Functions that do not support double backward should decorate their `backward()` method with `@once_differentiable` so that performing double backward

raises an error. If you'd like to support double backward, you can either recompute intermediaries based on the inputs during backward or return the intermediaries as the outputs of the custom Function. See the double backward tutorial for more details.

In `backward()`, saved tensors can be accessed through the `saved_tensors` attribute. Before returning them to the user, a check is made to ensure they weren't used in any in-place operation that modified their content.

Arguments can also be `None`. This is a no-op.

See Extending `torch.autograd` for more details on how to use this method.

Save given tensors for a future call to `jvp()`.

`save_for_forward` should be called at most once, in either the `setup_context()` or `forward()` methods, and all arguments should be tensors.

In `jvp()`, saved objects can be accessed through the `saved_tensors` attribute.

Arguments can also be `None`. This is a no-op.

See Extending `torch.autograd` for more details on how to use this method.

Set whether to materialize grad tensors. Default is `True`.

This should be called only from either the `setup_context()` or `forward()` methods.

If `True`, undefined grad tensors will be expanded to tensors full of zeros prior to calling the `backward()` and `jvp()` methods.

There are two ways to define the forward pass of an `autograd.Function`.

Override `forward` with the signature `forward(ctx, *args, **kwargs)`. `setup_context` is not

overridden. Setting up the ctx for backward happens inside the forward.

Override `forward` with the signature `forward(*args, **kwargs)` and override `setup_context`. Setting up the ctx for backward happens inside `setup_context` (as opposed to inside the forward)

See `torch.autograd.Function.forward()` and `Extending torch.autograd` for more details.

Define a formula for differentiating the operation with backward mode automatic differentiation.

This function is to be overridden by all subclasses. (Defining this function is equivalent to defining the `vjp` function.)

It must accept a context `ctx` as the first argument, followed by as many outputs as the `forward()` returned (None will be passed in for non tensor outputs of the forward function), and it should return as many tensors, as there were inputs to `forward()`. Each argument is the gradient w.r.t the given output, and each returned value should be the gradient w.r.t. the corresponding input. If an input is not a Tensor or is a Tensor not requiring grads, you can just pass None as a gradient for that input.

The context can be used to retrieve tensors saved during the forward pass. It also has an attribute `ctx.needs_input_grad` as a tuple of booleans representing whether each input needs gradient. E.g., `backward()` will have `ctx.needs_input_grad[0] = True` if the first input to `forward()` needs gradient computed w.r.t. the output.

Define the behavior for this `autograd.Function` underneath `torch.vmap()`.

For a `torch.autograd.Function()` to support `torch.vmap()`, you must either override this static method, or set `generate_vmap_rule` to True (you may not do both).

If you choose to override this staticmethod: it must accept

an `info` object as the first argument. `info.batch_size` specifies the size of the dimension being vmapped over, while `info.randomness` is the randomness option passed to `torch.vmap()`.

an `in_dims` tuple as the second argument. For each `arg` in `args`, `in_dims` has a corresponding `Optional[int]`. It is `None` if the `arg` is not a `Tensor` or if the `arg` is not being vmapped over, otherwise, it is an integer specifying what dimension of the `Tensor` is being vmapped over.

*`args`, which is the same as the `args` to `forward()`.

The return of the `vmap` staticmethod is a tuple of (`output`, `out_dims`). Similar to `in_dims`, `out_dims` should be of the same structure as `output` and contain one `out_dim` per output that specifies if the output has the vmapped dimension and what index it is in.

Please see `Extending torch.func with autograd.Function` for more details.